

上週裝的四樣，今天全用上

四樣不是各裝各的，今天串成一條線。

GitHub

放你的倉庫、給你網址

Student Pack

之後的雲端電腦

ChatGPT • Codex

路線 A' 的 agent

Antigravity

今天的主線

檢查點：四樣有缺的，開場五分鐘內舉手

今天的分工

它做十二件，你做六件

它動手，你判斷。

AI agent 替你做 12 件

查版本

裝 git 與 gh

建版本庫

寫 .gitignore

寫 commit 訊息

建 repo

推上去

開網址

寫一個 app

解釋 diff

找到那筆錯

救回來

你做 6 件

看計畫

按同意

自己登入

測一測

退回它

寫理由

檢查點：右邊六件，沒有人能替你做

夠了線

網址活著，今天就 100%

其餘都是往前，不是門檻。

今天 100%

網址打得開

貼進 e學苑

夠了線

往前

讀得懂 diff

救得回來

三條路線同分

檢查點：交件 = 兩行網址 + 一句理由

對號入座 · 3 分鐘

三條路，現在各自選一條

成品一樣、分數一樣，選定就照著走。

你手上有

走

用哪個 AI

筆電 + Antigravity 登得進

A

Antigravity

Antigravity 登不進去

A'

ChatGPT → Codex

沒帶筆電 / 什麼都沒裝

B

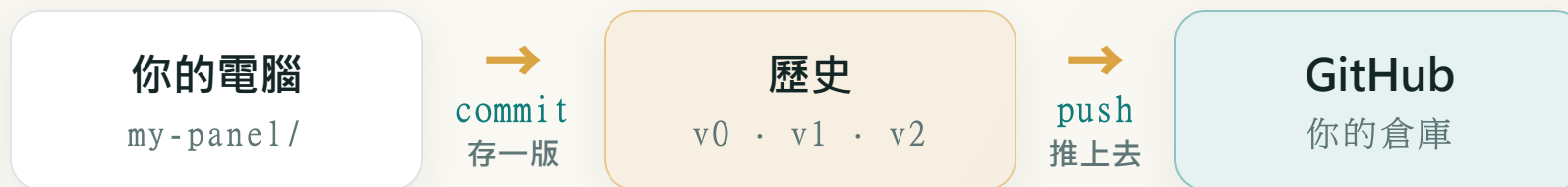
網頁版 AI + github.com

沒有上週那個檔 → 用起步包 v0，同分

👉 各自舉手：A、A'、B

Git 就是三個動作

它替一個資料夾記歷史，不管整台電腦。



diff = 兩版之間哪幾行變了：紅減號拿掉、綠加號新加

檢查點：commit ≠ push：存了，不等於網路上有了

畫面上，認得這五個地方

今天每一步都落在其中一個。

1
對話框
你貼提示詞

2
Implementation Plan
讀完按 Proceed / 不同意按 Review

3
跑指令前的詢問框
看一眼指令再按同意

4
diff / Walkthrough
它做了什麼、怎麼驗

5 設定裡那道關 **Request Review**

檢查點：字對不上就問它「這顆是什麼」

gh：讓 AI 替你按 GitHub 的按鈕

沒有它，AI 只能叫你自己去網站點。

```
gh repo create my-panel --public
```

→ GitHub 上多一個倉庫

```
gh auth login
```

→ 把代表你的鑰匙放進這台電腦

人自己做

檢查點：鑰匙不經過 AI，密碼也不給它

動手前，先檢查那道閘

切到 Always Proceed，它做什麼你都看不到。

Request Review

每條指令先問你

今天要這個

Proceed in Sandbox

沙箱裡自動跑

Always Proceed

不問你

今天不准

👉 打開設定，跟旁邊的人互相看一眼

卡 1 • 只看不動

卡 1：先叫它只看、不要動

Git 管的是一個資料夾；它站錯地方，後面每一步都錯。

貼這句

複製

我已經在這裡打開了資料夾 `my-panel`。請只查看、不要修改任何東西：告訴我這個資料夾的完整路徑、裡面有哪些檔案、這裡現在是不是 Git 版本庫，以及這台電腦有沒有裝 `git` 跟 `gh` (GitHub CLI)，各是什麼版本。

檢查點：回報的路徑，是不是你剛建的 `my-panel`



卡 2：讓它替你裝工具

每一條會動到電腦的指令，都會停下來等你按。

貼這句

複製

這台電腦如果沒有 git 或 gh (GitHub CLI)，請幫我安裝：Windows 用 winget (套件 Git.Git 與 GitHub.cli)，Mac 用 Homebrew (git 與 gh)。每一條指令執行前先告訴我它會做什麼，等我同意再跑。裝完後告訴我要不要重開終端機或重開這個程式才找得到新指令。不要幫我登入 GitHub，登入我自己來。

翻牌

```
winget install --id Git.Git -e
```

```
winget install --id GitHub.cli -e
```



先猜它會下哪兩條指令，再翻牌

登入這一步，人自己做

鑰匙不經過 AI——它不需要知道你的密碼。

1 `gh auth login`

2 GitHub.com → HTTPS → Yes 用瀏覽器登入

3 ABCD-1234 記下 8 碼，按 Enter

4 Authorize 瀏覽器貼上那 8 碼

~~把 8 碼或密碼貼進 AI 的對話框~~

檢查點：終端機出現 Logged in as 你的帳號

卡 3：確認登入、設定名字

repo 會公開，每一筆 commit 都刻著信箱。

貼這句

複製

請確認我已經登入 GitHub：執行 `gh auth status`，用白話告訴我結果。接著幫我設定 git 的作者名字與信箱：名字用我的 GitHub 帳號名；信箱請先用 `gh api user` 查出我的帳號 id 與 login，設成 GitHub 的 noreply 格式「id+login@users.noreply.github.com」，因為這個 repo 會公開、每一筆 commit 都帶著信箱。設定前先把要執行的指令列給我看，等我同意。

檢查點：信箱長得像 id+帳號@users.noreply.github.com

第一次鐘 · 休息 10 分鐘

休息十分鐘

Antigravity 跟終端機都不要關。

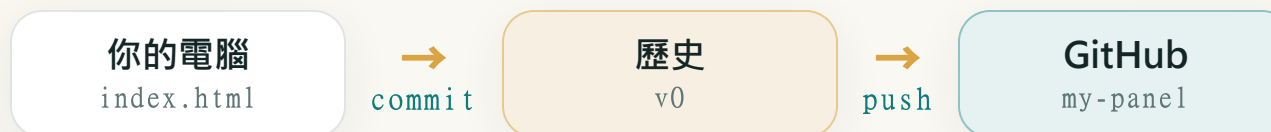
兩個都不要關

關掉 = 資料夾要重開、對話要重來

檢查點：回來接卡 4：第一版進倉庫

卡 4：第一版進倉庫、上 GitHub

commit 是存一版，push 是送上去——兩件事。



貼這句

複製

資料夾裡現在有 `index.html`，是我上週做的面板。請照順序做，每個會改東西的步驟執行前先列出來等我同意：

1. 替這個資料夾加上 Git 版本控制，分支叫 `main`。
 2. 建一個 `.gitignore`，排除 `.DS_Store` 與 `Thumbs.db`。
 3. 先把要收進第一版的檔案列給我，等我確認再 `commit`，訊息寫「`v0`：上週的面板放進版本控制」。
 4. 在我的 GitHub 建一個公開 repo 叫 `my-panel`，把本機的 `main` 推上去。
 5. 完成後給我 repo 網址與那筆 `commit` 的網址，我要自己打開確認。
- 公開之前提醒我檢查 `index.html` 裡有沒有真名、學號、電話。

檢查點：先把上週的檔改名 `index.html`，全小寫

檢查點 • GITHUB.COM

打開 github.com，要看到三件

AI 說「完成了」不算，你自己看到才算。



index.html
在最外層



1 commit
歷史有一筆



Public
公開



三件都看到的舉手，缺一件先別往下

卡 5：開網址

Pages 把 index.html 變成公開網址，通常一兩分鐘。

貼這句

複製

請替 GitHub 上的 my-panel 開啟 GitHub Pages，來源用 main 分支的根目錄。用 gh api 做，執行前先把指令給我看。開好後查詢並回報網站網址（html_url），並告訴我 GitHub 通常要等一兩分鐘才會亮。



檢查點：手機打得開 = 今天的 100%

卡 6：一張工單，做成真的 app

「界線」那欄最重要——它做錯，先看你少交代了什麼。

貼這句

複製

請照下面這張工單改 index.html。改之前先列出實作計畫（會改哪些檔、分幾步、哪裡你不確定），等我按同意再動手；做完給我改了什麼的摘要，以及我可以怎麼測。

【任務】把 v0 面板做成「我的一週小面板」

【輸出（我預期看到什麼）】

1. 最上面顯示今天日期，格式 YYYY-MM-DD。
2. 原本那顆按鈕還在；按下去的計數重新整理後還在（存在瀏覽器裡）。
3. 每天自動從 0 開始，但保留最近 7 天的數字，畫成 7 根長條圖，今天那根標出來。
4. 右上角一顆「深色／淺色」切換，重新整理後記得我的選擇。
5. 手機直式（寬 360px）不破版、不用左右捲。
6. 一顆「清除全部資料」按鈕，按下去先在畫面上問我一次再清（不用 alert）。

【界線（不放什麼）】

單檔 index.html，CSS 與 JavaScript 都寫在裡面；不連網、不用任何外部套件或字型；不用 alert／confirm／prompt；不放真名、學號、電話；不加動畫與音效；只改 index.html、不新增其他檔案。

👉 計畫出來先別按：它要改幾個檔？有沒有界線外的？

Proceed 還是 Review ?

它的計畫裡多了一條你沒交代的事。

Implementation Plan

1. 只改 index.html · 加上日期與 7 根長條圖
2. 用 localStorage 存每天的數字
3. 右上角加深色 / 淺色切換
4. 順便加一個進場動畫

Proceed

Review

👉 全班舉手：你按哪一顆？為什麼

驗收 · 五個邊角

跑起來，不等於做對

上週那張小表回來了：我預期 / 實際上 / 一樣嗎。

1

連按兩次
加 2 ?

2

重新整理
數字還在 ?

3

切深色再整理
還是深色 ?

4

縮到手機寬
不用左右捲 ?

5

日期格式
2026-09-21 ?

檢查點：五個都過才算做對

卡 6b：讀 diff，再存一版

diff 是你唯一親眼看見它做了什麼的地方。

```
document.getElementById("add").onclick = function () {  
    count = count + 1;  
-   document.getElementById("show").textContent = "今天喝了 " + count + " 杯";  
+   localStorage.setItem(KEY_TODAY, count);  
+   render();  
};
```

貼這句

複製

測過了，功能正確。請把這次修改的 diff 逐段列給我，用白話解釋紅色減號與綠色加號各是什麼；然後只把 index.html 存成第二個版本，commit 訊息寫「v1：記得住、七天長條圖、深色模式」，推上 GitHub。推完確認本機 main 跟 origin/main 一致。

👉 哪幾行新增？哪一行刪掉？為什麼刪？

第二次鐘 · 休息 10 分鐘

再休息十分鐘

一樣，兩個都不要關。

兩個都不要關

還沒 2 筆 commit 的，先補這件

檢查點：回來接卡 7：改壞再救回

卡 7：改壞了，救得回來

還沒存的錯用 `restore`；已經出去的錯用 `revert`。

`restore`

還沒 `commit` 的錯

一秒回到上一版

`revert`

已經 `push` 出去的錯

再加一筆改回去，歷史不改寫

v0 → v1 → 故意弄壞 → Revert

檢查點：歷史四筆都在，網址又活了

7a 弄壞

複製

請把 `index.html` 裡整段 `<script>...</script>` 刪掉並存檔，但不要 `commit`。然後顯示 `git diff`，告訴我現在工作區有什麼變化。

7b 救回

複製

我不要這個修改。請用 `git restore` 把 `index.html` 救回上一版，然後證明 `diff` 已經是空的、工作區乾淨。不要用 `reset`。

7c 再弄壞

複製

請再把 `<script>` 那一段刪掉一次，這次 `commit`（訊息寫「練習：故意刪掉 `script`」）並推上 GitHub。推完告訴我這筆 `commit` 的 `hash`。

7d 撤銷

複製

剛剛那筆是故意弄壞的。請用 `git log` 找到它，用 `git revert` 撤銷它（會新增一筆撤銷 `commit`，原本那筆留在歷史裡），然後推上 GitHub。全程不准 `reset`、`rebase`、`force push`。做完顯示最近四筆歷



卡 8：你的功能，退回它一次

拒絕得了它，你才是導演。

貼這句

複製

我想加一個功能：_____。請先列實作計畫，不要動手。

我不要 _____ X _____，因為 _____ Y _____。

沒點子？挑一個

「今天一句話」輸入框

「本週總杯數」一行字

長條圖換成七個圓點

改好、測過再貼這句

複製

可以了。請只把這次修改 commit，訊息寫「v2：_____」，推上 GitHub。

檢查點：那句理由，就是 e學苑 的第三行



收尾 · 分工

今天，它做的 / 你做的

同一份作品，兩種角色。

AI 替你做

裝工具

建倉庫

推上 GitHub

開網址

寫 app

救回來

你自己做

看計畫

按同意

自己登入

測五個邊角

退回一次

寫一句理由

檢查點：右邊六件，你每一件都做過了嗎

下課前 · 交件

交兩行網址，加一句理由

貼進 e學苑，當場自己點一次確認。

網站 你的帳號.github.io/my-panel/

倉庫 github.com/你的帳號/my-panel

理由 我退回它的那一次是什麼、為什麼

W5：兩人一組互相審對方的修改

檢查點：貼完當場自己點一次



課程網站